remote sensing

MDPI

Article

VE-LIOM: A Versatile and Efficient LiDAR-Inertial Odometry and Mapping System

Yuhang Gao and Long Zhao *

School of Automation Science and Electrical Engineering, Beihang University, Beiijng 100191, China
* Correspondence: flylong@buaa.edu.cn

Abstract: LiDAR has emerged as one of the most pivotal sensors in the field of navigation, owing to its expansive measurement range, high resolution, and adeptness in capturing intricate scene details. This significance is particularly pronounced in challenging navigation scenarios where GNSS signals encounter interference, such as within urban canyons and indoor environments. However, the copious volume of point cloud data poses a challenge, rendering traditional iterative closest point (ICP) methods inadequate in meeting real-time odometry requirements. Consequently, many algorithms have turned to feature extraction approaches. Nonetheless, with the advent of diverse scanning mode LiDARs, there arises a necessity to devise unique methods tailored to these sensors to facilitate algorithm migration. To address this challenge, we propose a weighted point-to-plane matching strategy that focuses on local details without relying on feature extraction. This improved approach mitigates the impact of imperfect plane fitting on localization accuracy. Moreover, we present a classification optimization method based on the normal vectors of planes to further refine algorithmic efficiency. Finally, we devise a tightly coupled LiDAR-inertial odometry system founded upon optimization schemes. Notably, we pioneer the derivation of an online gravity estimation method from the perspective of $\mathcal{S}^2$ manifold optimization, effectively minimizing the influence of gravity estimation errors introduced during the initialization phase on localization accuracy. The efficacy of the proposed method was validated through experimentation employing various LiDAR sensors. The outcomes of indoor and outdoor experiments substantiate its capability to furnish real-time and precise localization and mapping results.

Keywords: LiDAR; IMU; sensor fusion

check for updates

Citation: Gao, Y.; Zhao, L. VE-LIOM: A Versatile and Efficient LiDAR-Inertial Odometry and Mapping System. *Remote Sens.* **2024**, *16*, 2772. https://doi.org/10.3390/rs16152772

Academic Editor: Gino Dardanelli

Received: 4 June 2024
Revised: 15 July 2024
Accepted: 27 July 2024
Published: 29 July 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Simultaneous localization and mapping (SLAM) has advanced quickly in recent years and now serves as the foundation for path planning and autonomous navigation by intelligent mobile robots. SLAM can be classified into two categories of algorithms based on vision and LiDAR, depending on the type of sensor being used. Most vision-based algorithms can only produce sparse maps since visual SLAM is vulnerable to changes in light and viewpoint and limited by computational resources. Compared with vision sensors, LiDAR has a larger measurement range and can directly measure the distance to the target; it can capture the fine details of the scene and is widely used in autonomous driving, mobile robotics, and mapping [1–3].

Based on their scanning mode, LiDARs can be categorized into two types: mechanical LiDAR and solid-state LiDAR. Mechanical LiDAR employs rotating high-frequency laser arrays to capture environmental data, typically offering a 360° field of view (FoV) in the horizontal direction but with lower vertical resolution. Examples include the Velodyne Puck (https://velodynelidar.com/) and Robosense RS-LiDAR-16 (https://www.robosense.cn/rslidar/RS-LiDAR-16). These sophisticated sensors are commonly utilized in self-driving vehicles. Conversely, solid-state LiDAR sensors like the Intel RealSense L515 (https://www.intelrealsense.com/lidar-camera-l515/) and Livox (https://www.livoxtech.com/) series

have emerged as cost-effective alternatives, particularly suitable for small-scale robots due to their lightweight design and high performance. However, owing to distinct operational modes, most existing LiDAR odometry algorithms are tailored for specific sensor types, with only a few capable of accommodating multiple LiDAR types. Furthermore, given that LiDAR scans typically comprise thousands of points, many algorithms need to extract features from the raw point cloud to ensure computational efficiency. This process involves analyzing the smoothness of candidate points and their surrounding environments to identify edge and plane features. Feature extraction from point clouds significantly reduces the volume of data to be processed and enhances the efficiency of subsequent localization and mapping processes [4–6]. While these techniques enhance computational efficiency, they may struggle in feature-scarce scenarios. Furthermore, in environments lacking distinct features, such as tunnels, standalone LiDAR odometry systems are prone to performance degradation [7]. The acquisition of a frame of point cloud data through continuous scanning by LiDAR can introduce motion distortion, and relying solely on LiDAR for positional estimation can result in significant drift. To address this limitation, many algorithms incorporate IMU sensors, capable of measuring the acceleration and angular velocity of ego-motion with high update frequencies [6,8]. IMU measurements serve two main purposes: firstly, they can compensate for motion distortion in LiDAR scans, and secondly, the motion predictions derived from IMU data can serve as initial values for odometry, expediting optimization convergence. The fusion of LiDAR and IMU data typically falls into two categories: loosely coupled and tightly coupled methods. Loosely coupled algorithms process LiDAR and IMU measurements independently to derive motion constraints, which are then fused subsequently. While this approach offers computational efficiency, it may lead to information loss. In contrast, tightly coupled methods directly integrate LiDAR and IMU measurements through joint optimization, enabling higher positioning accuracy. In the context of optimization-based LiDAR-inertial odometry systems, while most systems estimate the gravity vector only during system initialization, prolonged operation may lead to drift in the estimated gravity vector.

To address the aforementioned challenges, we present a versatile and efficient tightly coupled LiDAR-inertial odometry and mapping framework. The contributions of this work are outlined as follows:

(1) We present a weighted point-to-plane direct registration method for raw point clouds. This approach eliminates the need for feature extraction, allowing the algorithm to seamlessly adapt to various types of LiDAR. Furthermore, it helps mitigate pose estimation errors resulting from imperfect plane fitting.
(2) We construct an optimization-based, tightly coupled LiDAR-inertial odometry system. To accelerate optimization convergence, we propose a multi-step approach that categorizes fitted planes into three classes according to their normal vectors, with each class involved in distinct optimization steps. Furthermore, we incorporate gravity as a system state for online estimation, marking the first optimization-based online gravity estimation system to our knowledge.
(3) The proposed framework undergoes comprehensive validation using datasets encompassing various sensor types, platforms, and environmental conditions. Our findings substantiate that the proposed method achieves real-time and precise localization across multiple LiDAR types in diverse indoor and outdoor settings.

The remainder of this paper is structured as follows: Section 2 delves into the pertinent literature. Section 3 provides a comprehensive overview of the complete system pipeline. The experimental results are detailed in Section 4, followed by concluding remarks in Section 5.

2. Related Works

In recent years, numerous studies focusing on LiDAR applications have surfaced. In this review, we specifically concentrate on LiDAR-based SLAM methodologies, encompassing both standalone LiDAR methods and LiDAR-IMU fusion techniques. Based on

the type of sensor utilized by the algorithms, we categorize the relevant research into two groups: mechanical LiDAR algorithms and solid-state LiDAR algorithms. These categories will be delineated in the subsequent sections.

2.1. Mechanical LiDAR-(Inertial) Odometry

The LOAM [4] method, regarded as a seminal work in 3D LiDAR SLAM, has garnered widespread acclaim for its exemplary performance. It introduces an innovative feature extraction technique that selects edge and plane points based on local surface smoothness, minimizing the distance between feature points and their correspondences through optimization to derive odometry. Given that a LiDAR scan typically comprises thousands of points, the inclusion of feature extraction significantly reduces the computational burden of direct point cloud matching, thereby ensuring real-time odometry performance.

Building upon LOAM, LeGO-LOAM [5] integrates a ground plane in its segmentation and optimization stages, achieving comparable or superior accuracy to LOAM while reducing computational overhead. This adaptation is particularly suitable for unmanned ground vehicles (UGVs) with limited computational resources. In contrast, F-LOAM [9] proposes a non-iterative, two-stage distortion compensation method to further reduce computational costs. By assuming uniform sensor motion for a brief period and subsequently correcting distortion through motion prediction, F-LOAM optimizes predicted poses to refine distortion and update undistorted features into the map.

Pure LiDAR odometry may encounter challenges in feature-sparse scenarios and exhibit drift in large-scale testing. To address these limitations, both LIO-mapping [10] and LIO-SAM [6] introduce preintegration [11] techniques from visual-inertial odometry. By leveraging IMU measurements between consecutive LiDAR scans, these methods correct motion distortion and predict motion increments to expedite optimization convergence, aiming for more precise and reliable localization results. Notably, LIO-mapping optimizes all states within a local window using a rotation-constrained algorithm, albeit sacrificing real-time performance due to increased computational demands. Conversely, LIO-SAM maintains real-time performance by incorporating key frames into the optimization process to balance map density and computational efficiency, supplemented by loop closure detection to mitigate drift. It is noteworthy that LIO-SAM requires a 9-axis IMU to provide prior attitude for scan matching.

In addition to optimization-based approaches, LINS [12] introduces a filter-based framework and an iterated error-state Kalman filter with a robocentric formulation, ensuring stable 6-DoF pose estimation in challenging environments, albeit limited to ground vehicles. It is important to note that these algorithms rely on the feature extraction scheme from LOAM, rendering them suitable only for multi-line rotating LiDARs.

2.2. Solid-State LiDAR-(Inertial) Odometry

Solid-state LiDAR has garnered increasing attention recently due to its advantages of low cost, light weight, and high performance, rendering it well-suited for constructing affordable autonomous mobile robots. In comparison to mechanical LiDAR, solid-state LiDAR boasts a higher update frequency and angular resolution but typically features a smaller field of view (FoV). It is often scanned using irregular scanning patterns in a non-repetitive manner to maximize environmental coverage.

SSL-SLAM [13] is a pure LiDAR SLAM algorithm specifically designed for solid-state LiDAR. However, it has only been validated with the Intel RealSense L515 LiDAR, which is capable of directly outputting RGB-colored point clouds, facilitating real-time RGB-colored map construction. LOAM Livox [14] introduces a framework tailored for small FoVs and irregular LiDAR scanning. To address the challenge of limited features arising from a small FoV, reflectivity is utilized as an additional constraint. Both SSL-SLAM and LOAM Livox are pure LiDAR odometry methods, which may suffer performance degradation when facing challenging movements, such as those encountered with handheld devices.

LIO-Livox [15] implements a robust LiDAR-inertial odometry system specifically designed for Livox LiDAR. The system extracts widely distributed and uniform feature points, including corner points, surface points, and irregular points, and removes dynamic objects to enhance robustness. It can operate at speeds of about 80 km/h on highways. LiLi-OM [16] presents an optimization-based LiDAR-inertial odometry and mapping algorithm featuring a novel feature extraction method designed for LiDARs with irregular scanning patterns. Its generic backend is also applicable to mechanical LiDAR. FAST-LIO [8] adopts an iterated extended Kalman filter framework akin to LINS. However, its back-propagation process effectively compensates for motion distortion within the point cloud. Additionally, a new formula for computing the Kalman gain reduces the computational burden, enhancing efficiency and accuracy compared to LINS. Building upon FAST-LIO, FAST-LIO2 [17] introduces an ikd-Tree [18] data structure that allows incremental updates and dynamic rebalancing. This approach registers raw points directly without feature extraction and is compatible with various LiDARs. Point-LIO [19] presents a robust and high-bandwidth LiDAR-inertial odometry system capable of estimating extremely aggressive robotic motions. Its innovations include a point-by-point LIO framework that updates the state at each LiDAR point measurement and a stochastic process-augmented kinematic model, providing accurate localization and reliable mapping under high-frequency conditions and severe vibrations.

3. Methods

3.1. System Overview

The overview of the proposed framework is depicted in Figure 1. The system adopts an optimization-based, tightly coupled framework for LiDAR and IMU integration. To accommodate various LiDAR scanning modes, the point cloud feature extraction module is omitted in favor of direct raw point cloud matching. Initially, the system receives time-synchronized LiDAR point clouds and IMU measurement data. IMU preintegration is then employed for motion compensation of the point cloud. The deskewed point cloud undergoes weighted scan-to-map registration. Based on the locally fitted plane normal vectors during registration, the point cloud is categorized into horizontal plane points, vertical plane points, and general points. The registration residuals of horizontal and vertical plane points are used to compute different components of the 6-DoF pose, termed the first optimization. The optimization results from the first step serve as initial values for the joint optimization of the registration residuals of general points and the IMU preintegration residuals, which include gravity constraints. Compared to predictions from a constant velocity model or an IMU motion model, the optimized results are closer to the true values, facilitating rapid convergence in the second optimization. The estimated odometry from the second optimization updates the local map with the current scan. The updated map is then used for matching the next scan. The following sections provide a detailed exposition of the proposed system.

Before delving into specifics, we establish the notation and frames utilized throughout this paper. We define the IMU frame as the body frame, denoted as $(\cdot)^I$. The first IMU frame is considered the world frame, denoted as $(\cdot)^W$. $\mathbf{g}^W = \left[g_x, g_y, g_z\right]^T$ represents the gravity vector in the world frame. We assume a rigid link between the IMU and the LiDAR.

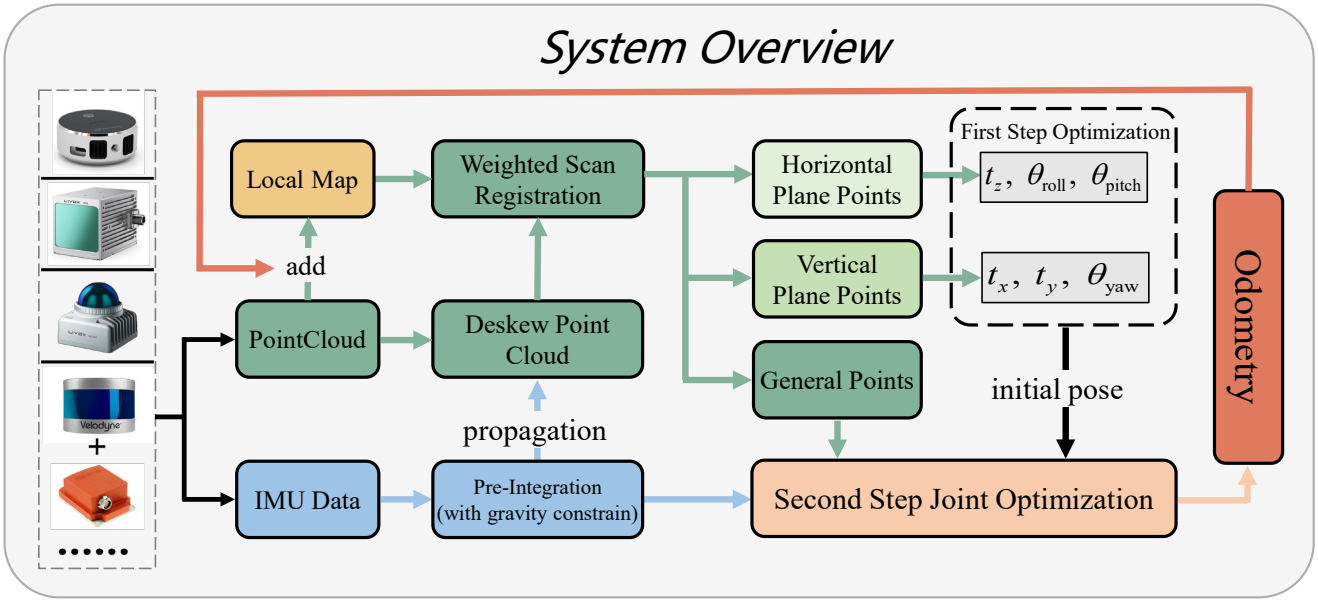


Figure 1. System overview of the proposed framework.

3.2. IMU Preintegration

The measurements of the IMU are modeled as

$$\begin{aligned}\hat{\boldsymbol{\omega}}_t &= \boldsymbol{\omega}_t + \mathbf{b}_{\omega_t} + \mathbf{n}_{\omega} \\ \hat{\mathbf{a}}_t &= \mathbf{a}_t + \mathbf{b}_{\mathbf{a}_t} + \mathbf{R}_W^{I_t}\mathbf{g}^W + \mathbf{n}_{\mathbf{a}}\end{aligned} \tag{1}$$

where $\hat{\boldsymbol{\omega}}_t$ and $\hat{\mathbf{a}}_t$ are the raw IMU measurements in the body frame at time t. The IMU measurements are affected by bias $\mathbf{b}_t$ and additive white noise $\mathbf{n}_{\omega}$ and $\mathbf{n}_{\mathbf{a}}$. $\mathbf{R}_W^I$ is the rotation matrix from the world frame to the body frame. Assuming that the position, velocity, rotation, and IMU measurements of the platform at time t are known, and the platform keeps a constant velocity for a brief period of time. Then, we can predict the position, velocity, and orientation of the platform at time $t+1$:

$$\begin{aligned}\mathbf{p}_{t+1} &= \mathbf{p}_t + \mathbf{v}_t\Delta t + \frac{1}{2}\mathbf{g}^W\Delta t^2 + \frac{1}{2}\mathbf{R}_t(\hat{\mathbf{a}}_t - \mathbf{b}_{\mathbf{a}_t} - \mathbf{n}_{\mathbf{a}})\Delta t^2 \\ \mathbf{v}_{t+1} &= \mathbf{v}_t + \mathbf{g}^W\Delta t + \mathbf{R}_t(\hat{\mathbf{a}}_t - \mathbf{b}_{\mathbf{a}_t} - \mathbf{n}_{\mathbf{a}})\Delta t \\ \mathbf{R}_{t+1} &= \mathbf{R}_t\mathrm{Exp}((\hat{\boldsymbol{\omega}}_t - \mathbf{b}_{\omega_t} - \mathbf{n}_{\omega})\Delta t)\end{aligned} \tag{2}$$

where Δt is the time step from time t to time $t+1$, and $\mathbf{p}$, $\mathbf{v}$, and $\mathbf{R}$ represent displacement, velocity, and rotation matrix from the body frame to the world frame, respectively. To avoid recalculating the above equation when the linearization point changes at time t, the preintegration method proposed in [11] is introduced, that is, to calculate the relative motion from time t to time $t+1$,

$$\begin{aligned}\Delta\mathbf{p}_{t+1}^t &= \mathbf{R}_t^T\left(\mathbf{p}_{t+1} - \mathbf{p}_t - \mathbf{v}_t\Delta t - \frac{1}{2}\mathbf{g}^W\Delta t^2\right) \\ \Delta\mathbf{v}_{t+1}^t &= \mathbf{R}_t^T\left(\mathbf{v}_{t+1} - \mathbf{v}_t - \mathbf{g}^W\Delta t\right) \\ \Delta\mathbf{R}_{t+1}^t &= \mathbf{R}_t^T\mathbf{R}_{t+1}.\end{aligned} \tag{3}$$

The incorporation of preintegration not only enhances computational efficiency but also establishes a constraint for sensor fusion, as elaborated in Section 3.4. To delve into the specific derivation of preintegration, readers are encouraged to refer to [11] due to space limitations.

3.3. Weighted Point Cloud Registration and Classification

The LiDAR point cloud is acquired through continuous scanning, where the movement of the platform during scanning can introduce distortions to the point cloud. As depicted in Figure 2, the LiDAR continuously moves from the start to the end of the scanning process, represented as ΔT. Points originally situated on the same plane on the object (depicted in black) may appear as non-coplanar points (depicted in blue). To achieve precise results in point cloud registration, we utilize IMU measurements to generate motion corrections. However, due to the different sampling frequencies of the sensors, a certain number of LiDAR points may be distributed between two consecutive IMU measurements. Let t_s and t_e denote the start and end times of a LiDAR scan, respectively. For a point $\mathbf{P}_m$ measured at time m, we identify the two closest IMU measurements in time: $\mathbf{I}_j$ and $\mathbf{I}_{j+1}$. Consequently, the platform's position at time m can be predicted:

$$\begin{aligned}\mathbf{p}_m &= \mathbf{p}_j + \mathbf{v}_j\Delta t + \frac{1}{2}\mathbf{g}\Delta t^2 + \frac{1}{2}\mathbf{R}_j\left(\hat{\mathbf{a}}_j - \mathbf{b}_{\mathbf{a}_j} - \mathbf{n}_{\mathbf{a}}\right)\Delta t^2 \\ \mathbf{R}_m &= \mathbf{R}_j \mathrm{Exp}\left(\left(\hat{\omega}_j - \mathbf{b}_{\omega_j} - \mathbf{n}_{\omega_j}\right)\Delta t\right)\end{aligned} \tag{4}$$

where Δt represents the time interval between $\mathbf{P}_m$ and $\mathbf{I}_j$. To ensure consistency in the timing of the point cloud within the same scan, all scattered points are projected to the end time of the scan:

$$\begin{aligned}\Delta\mathbf{p}_m &= \mathbf{p}_m - \mathbf{p}_e \\ \mathbf{P}_m^{L_e} &= \mathbf{T}_L^{I\,-1}\mathbf{T}_{I_e}^{W\,-1}\left(\mathbf{T}_m^W\mathbf{T}_L^I\mathbf{P}_m + \Delta\mathbf{p}_m\right)\end{aligned} \tag{5}$$

where $\mathbf{T}_L^I$ denotes the extrinsic transformation between the LiDAR and IMU, $\mathbf{p}_e$ represents the platform position at the end of the scan in the world frame, and $\mathbf{T}_m^W = [\mathbf{R}_m \mid \mathbf{p}_m]$ signifies the transformation from the body frame at time m to the world frame. The corner mark I_e denotes the body frame at the end of the scan. The undistorted point $\mathbf{P}_m^{L_e}$ is subsequently utilized for scan-to-map registration.

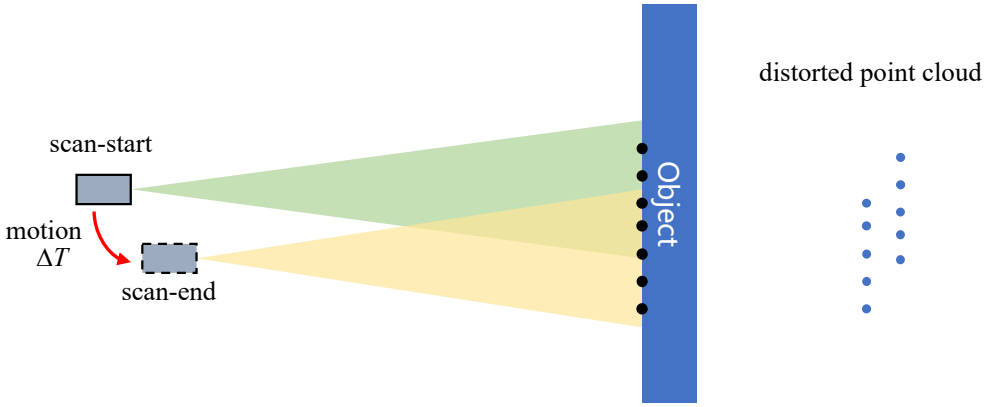


Figure 2. Continuous scanning of LiDAR and distorted point cloud.

We focus on the development of a LiDAR-inertial odometry and mapping system tailored for various LiDAR types. To cater to diverse scan patterns, we employ direct matching on raw points. This method not only leverages intricate environmental features to boost accuracy but also simplifies the process by eliminating manual feature extraction, thereby reducing preprocessing time for LiDAR point clouds. As illustrated in Figure 3, the current point is initially represented in the world frame. Subsequently, the system searches for the five closest points to the current point within the local map. A plane is then fitted through these five points. If the distance between each point involved in plane fitting and the fitted plane falls below the pre-set threshold, the current point is selected as the point to be matched and is subsequently used for odometry estimation. Let $\mathbf{n}_i = [a, b, c, d]$

denote the coefficients of the fitted plane. The distance from point $\mathbf{P}_i = [x_i, y_i, z_i]$ to the fitted plane is calculated as follows:

$$d_i = \frac{|ax_i + by_i + cz_i + d|}{\sqrt{a^2 + b^2 + c^2}}. \tag{6}$$

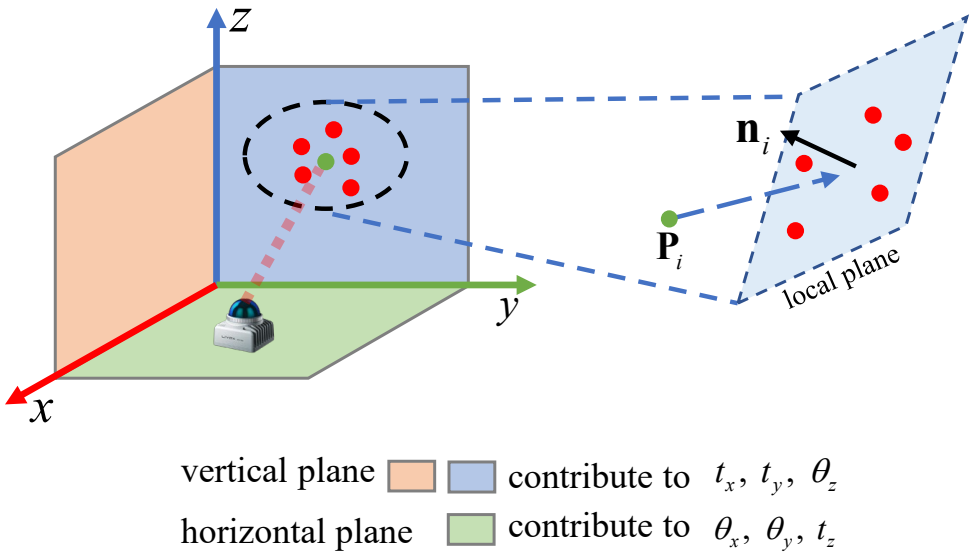


Figure 3. Scan-to-map registration. The local map is searched for the five closest points (depicted in red) to the scan point (depicted in green) and a plane is fitted with these searched points.

Traditional point-to-plane matching, as employed by FAST-LIO2 [17] and Point-LIO [19], focuses solely on optimizing geometric distances without considering the local geometric distribution of individual feature points. In practical applications, the accuracy of plane fitting is crucial for point cloud matching. Generally, the smaller the average distance from the points involved in plane fitting to the fitted plane, the more accurately the fitted plane can represent the local geometric distribution. Therefore, we introduce a weighting function to further balance the matching process. This weighting function is designed to adjust the importance of each feature point in the matching process based on the distance from the point to the plane, thereby better reflecting the characteristics of the local geometric distribution. To reduce computational costs, we reuse information from previously fitted planes to determine the weighting function. Specifically, for the mean distance d_{mean} from the points involved in plane fitting to the plane, the weight W of the fitted plane can be defined as

$$W(\mathbf{P}_m) = \frac{\exp\left(-d_{\text{mean}}^{(m)}\right)}{\sum_{\mathbf{P}^{(i)} \in \mathcal{S}_k} \exp\left(-d_{\text{mean}}^{(i)}\right)} \tag{7}$$

where $\mathcal{S}_k$ is the point cloud at the kth scan.

As a LiDAR scan typically consists of thousands of points, and with real-time requirements in mind, we commence by downsampling the raw point cloud. Subsequently, we traverse the downsampled points to perform plane fitting. Concurrently, we utilize an ikd-Tree [18] to manage the map and conduct efficient nearest neighbor searches. The ikd-Tree proves to be a highly effective data structure, allowing for incremental updates and dynamic data balancing. For more in-depth information, readers are encouraged to consult [18].

Inspired by LeGO-LOAM [5], we categorize the fitted planes into three classes based on their normal vectors. Specifically, through the fitting of the plane's normal vector $\bar{\mathbf{n}}_i = [a, b, c]$, if the angle between $\bar{\mathbf{n}}_i$ and the normal vector of the horizontal plane $\bar{\mathbf{n}} = [0, 0, 1]$ is approximately 0°, the current point is labeled as a candidate horizontal

plane point. Similarly, if this angle is close to 90°, the current point is identified as a candidate vertical plane point. Otherwise, the current point is categorized as a general point. The registration residuals of points labeled as horizontal planes are employed in solving the $\left[\theta_x,\ \theta_y,\ t_z\right]$ part of the 6-DoF pose, while the residuals of points labeled as vertical planes contribute to solving the $\left[t_x,\ t_y,\ \theta_z\right]$ components of the 6-DoF pose. A detailed explanation is provided in Section 3.4. It is worth mentioning, LeGO-LOAM considers only ground horizontal constraints and requires additional preprocessing time for ground segmentation. Our method leverages normal vectors during the point cloud registration process for classification, eliminating extra processing time and extending classification beyond just ground points. This approach not only saves computational costs but also enhances the adaptability and robustness of the system across various types of environments.

While we divide points into vertical, horizontal, and general categories, it is important to clarify that our method is designed to adapt to various environmental conditions, including those lacking distinct vertical or horizontal features. In such scenarios, the majority of points may indeed be classified as general points. However, the subsequent point cloud matching and optimization calculations remain effective.

3.4. Odometry Estimation

Drawing inspiration from visual-inertial odometry [20,21], we adopt a tightly coupled, optimization-based approach to construct our proposed LiDAR-inertial odometry system. While previous optimization-based LiDAR-inertial odometry methods [6,10,16] have been explored, few address the gravity vector as an estimated state. Typically, these methods align the gravity vector with the world frame upon system initialization and consider it as a known parameter thereafter. However, it is important to note that the gravity estimation made during initialization may drift over time. To address this issue, we include the gravity vector as one of the states to be optimized in this study. The complete state vector is defined as follows:

$$\begin{aligned} \boldsymbol{\mathcal{O}} &= \left[\mathbf{x}_0,\ \mathbf{x}_1,\ \cdots \mathbf{x}_n,\ \mathbf{g}^W\right] \\ \mathbf{x}_k &= \left[\mathbf{R}_{b_k}^W, \mathbf{p}_{b_k}^W, \mathbf{v}_{b_k}^W,\ \mathbf{b}_k\right],\ k \in [0, n] \end{aligned} \tag{8}$$

where $\mathbf{x}_k$ represents the IMU state at the kth LiDAR scan end time, encompassing the orientation, position, and velocity of the IMU in the world frame, along with IMU bias in the body frame. The parameter n denotes the total number of LiDAR scans.

We design a two-step optimization approach to achieve the maximum a posteriori (MAP) estimate of the system state by solving two nonlinear least squares problems.

3.4.1. First Step: Optimization

The first-step optimization involves computing the initial 6-DoF pose values based on the classification results of scanned points. In this phase, we select scanned points labeled as horizontal planes and vertical planes, with the quantities denoted by v and h, respectively. Given that horizontal plane points constrain only $\left[\theta_x,\ \theta_y,\ t_z\right]$, and vertical plane points constrain only $\left[t_x,\ t_y,\ \theta_z\right]$, during optimization, the unconstrained parameters can be fixed to enhance computational efficiency. The optimization aims to minimize the following cost function based on the registration residuals of the scanned points:

$$\min_{\mathbf{T}} \frac{1}{2}\left\{ \left\|\mathbf{r}_{\mathcal{L}_\mathrm{V}}(m, \mathbf{T})\right\|_{fix}^2 + \left\|\mathbf{r}_{\mathcal{L}_\mathbf{H}}(m, \mathbf{T})\right\|_{fix}^2 \right\} \tag{9}$$

where $\mathbf{T} = \left[\theta_x,\ \theta_y,\ \theta_z,\ t_x,\ t_y,\ t_z\right]$ represents the six DoFs of the pose expressed in Euler angles for rotation components. $\|\cdot\|_{fix}^2$ denotes the squared norm, but fixing some parameters during optimization, where a subset of parameters is held constant. $\mathbf{r}_{\mathcal{L}_\mathrm{V}}(m, \mathbf{T})$ and $\mathbf{r}_{\mathcal{L}_\mathbf{H}}(m, \mathbf{T})$ correspond to the sum of distances from each point in the current scan to the planes fitted in the local map during the registration of points labeled as vertical

and horizontal planes, respectively. The Jacobian matrix of the point-to-plane residual is defined by

$$\begin{aligned}\mathbf{J} &= \frac{\partial\left(\bar{\mathbf{n}}_i^T\mathbf{p}_i + d\right)}{\partial\mathbf{T}} = \frac{\partial\left(\bar{\mathbf{n}}_i^T\mathbf{p}_i + d\right)}{\partial\mathbf{p}_i}\cdot\frac{\partial\mathbf{p}_i}{\partial\mathbf{T}} \\ &= \bar{\mathbf{n}}_i^T\cdot\frac{\partial\mathbf{p}_i}{\partial\mathbf{T}}\end{aligned} \tag{10}$$

where

$$\frac{\partial\mathbf{p}_i}{\partial\mathbf{T}} = \begin{bmatrix} \frac{\partial p_x}{\partial\theta_x} & \frac{\partial p_x}{\partial\theta_y} & \frac{\partial p_x}{\partial\theta_z} & 1 & 0 & 0 \\ \frac{\partial p_y}{\partial\theta_x} & \frac{\partial p_y}{\partial\theta_y} & \frac{\partial p_y}{\partial\theta_z} & 0 & 1 & 0 \\ \frac{\partial p_z}{\partial\theta_x} & \frac{\partial p_z}{\partial\theta_y} & \frac{\partial p_z}{\partial\theta_z} & 0 & 0 & 1 \end{bmatrix}. \tag{11}$$

The derivation of $\frac{\partial p}{\partial\theta}$ is evident. Owing to space constraints, the specific form is not provided in this context.

The pose optimization in this step is initialized using IMU preintegration. In contrast to a constant velocity motion model, this method enables a more accurate prediction of platform motion. Consequently, the initial values for optimization are closer to the optimal solution.

3.4.2. Second Step: Joint Optimization

The aim of the second optimization is to minimize the residual from IMU preintegration and the registration residual of points labeled as general points in the LiDAR scan. The associated cost function is formulated as follows:

$$\min_{\mathbf{x}}\frac{1}{2}\left\{\sum_{m\in\mathcal{L}}\left\|\mathbf{r}_{\mathcal{L}}(m,\mathbf{x})\right\|^2_{\mathbf{C}_L} + \left\|\mathbf{r}_{\mathcal{B}}(z^{\beta}_{\beta+1},\mathbf{x})\right\|^2_{\mathbf{C_B}}\right\} \tag{12}$$

where $\mathbf{r}_{\mathcal{L}}(m,\mathbf{x})$ and $\mathbf{r}_{\mathcal{B}}(z^{\beta}_{\beta+1},\mathbf{x})$ denote the weighted registration residuals of the LiDAR scan and the preintegration residuals of the IMU with the gravity constraint, respectively. $\mathcal{L}$ is the set of the current scan labeled as a general point. $\mathbf{C_L}$ and $\mathbf{C_B}$ represent the covariance matrices of the LiDAR and IMU residual terms. The initial values of parameters $\mathbf{R}$ and $\mathbf{p}$ in this optimization step are dictated by the outcomes of the first optimization step, given by $\mathbf{T} = \left[\theta_x,\ \theta_y,\ \theta_z,\ t_x,\ t_y,\ t_z\right]$:

$$\begin{aligned}\mathbf{R} &= R(\theta_z)R(\theta_y)R(\theta_x) \\ \mathbf{p} &= \left[t_x,\ t_y,\ t_z\right]\end{aligned} \tag{13}$$

where $R(\cdot)$ is the function to convert Euler angles into a rotation matrix.

The LiDAR residuals offer geometric constraints for pose optimization of the current LiDAR frame. For each point $\mathbf{P}_m$ in the current scan, the residual is defined as the distance from it to its corresponding fitted plane $\mathbf{n}_m$ in the local map:

$$\mathbf{r}_{\mathcal{L}}(m,\mathbf{x}_j) = W(\mathbf{P}_m)\cdot\mathbf{n}_m\cdot\left(\mathbf{R}^W_{I_j}\left(\mathbf{R}^I_L\mathbf{P}_m + \mathbf{p}^I_L\right) + \mathbf{p}^W_{I_j}\right) \tag{14}$$

where $W(\mathbf{P}_m)$ is defined in Equation (7).

The IMU preintegration provides a relative motion constraint for two consecutive frames of LiDAR scans. To optimize the gravity vector, we extend gravity variation to the IMU measurement residual:

$$\mathbf{r}_{\mathcal{B}}(z^{I_\beta}_{I_{\beta+1}}, \mathbf{x}_{\beta+1}) = \begin{bmatrix} \delta\boldsymbol{\alpha}^{I_\beta}_{I_{\beta+1}} \\ \delta\boldsymbol{\beta}^{I_\beta}_{I_{\beta+1}} \\ \delta\boldsymbol{\theta}^{I_\beta}_{I_{\beta+1}} \\ \delta\mathbf{b}_{\mathbf{a}} \\ \delta\mathbf{b}_{\boldsymbol{\omega}} \\ \delta\mathbf{g} \end{bmatrix} = \begin{bmatrix} \mathbf{R}^W_{I_\beta}\left(\mathbf{p}^W_{I_{\beta+1}} - \mathbf{p}^W_{I_\beta} + \frac{1}{2}\mathbf{g}^W\Delta t^2 - \mathbf{v}^W_{I_\beta}\Delta t\right) - \Delta\mathbf{p}^\beta_{\beta+1} \\ \mathbf{R}^W_{I_\beta}\left(\mathbf{v}^W_{I_{\beta+1}} + \mathbf{g}^W\Delta t - \mathbf{v}^W_{I_\beta}\right) - \Delta\mathbf{v}^\beta_{\beta+1} \\ Log\left({\mathbf{R}^W_{I_{\beta+1}}}^T\mathbf{R}^W_{I_\beta}\Delta\mathbf{R}^\beta_{\beta+1}\right) \\ \mathbf{b}_{\mathbf{a}_{\beta+1}} - \mathbf{b}_{\mathbf{a}_\beta} \\ \mathbf{b}_{\boldsymbol{\omega}_{\beta+1}} - \mathbf{b}_{\boldsymbol{\omega}_\beta} \\ \mathbf{g}^W_{\beta+1} - \mathbf{g}^W_\beta \end{bmatrix} \tag{15}$$

where $\Delta\mathbf{p}^\beta_{\beta+1}$, $\Delta\mathbf{v}^\beta_{\beta+1}$, and $\Delta\mathbf{R}^\beta_{\beta+1}$ represent the IMU preintegration as defined in Equation (3). For a detailed derivation of the Jacobian of each residual term of IMU preintegration, we refer readers to [11]. This paper primarily focuses on optimizing and updating the gravity vector. Despite being a three-dimensional vector, gravity exhibits only two degrees of freedom owing to its known magnitude, meaning it lies on a unit sphere. Simply optimizing the gravity vector as a three-dimensional entity necessitates normalization after each optimization, often leading to unstable system outcomes. Hence, we propose optimizing gravity on the mathematical $\mathcal{S}^2$-space [22], offering the advantage of a closed optimization operation. This ensures that the optimized gravity consistently resides on a sphere without requiring redundant normalization procedures. Let $\mathbf{x} = [x, y, z]^T$ and $\mathbf{u} \in \mathbb{R}^2$ denote the variables on the manifold $\mathcal{S}^2$ and the corresponding tangent plane, respectively. Moreover, let $R_{\mathbf{x}}(\delta)$ denote the local map: $\mathcal{S}^2 \mapsto \mathbb{R}^3$. Then, we have

$$\widehat{\mathbf{x}} = \mathbf{x} \oplus \mathbf{u} = \mathrm{Exp}(R_{\mathbf{x}}(\mathbf{u}))\mathbf{x} \tag{16}$$

where

$$R_{\mathbf{x}}(\mathbf{u}) = B_{\mathbf{x}}\mathbf{u} = \begin{bmatrix} 1 - \frac{x^2}{1+z} & -\frac{xy}{1+z} \\ -\frac{xy}{1+z} & 1 - \frac{y^2}{1+z} \\ -x & -y \end{bmatrix}\mathbf{u}. \tag{17}$$

Then, we can derive the Jacobian matrix of the gravity vector as follows:

$$\frac{\partial\mathbf{r}_{\mathcal{B}}}{\partial\delta\mathbf{g}} = \begin{bmatrix} -\frac{1}{2}{\mathbf{R}^W_{I_{\beta+1}}}^T\left\lfloor\mathbf{g}^W_\beta\Delta t^2\right\rfloor_\times B_{\mathbf{g}} \\ -{\mathbf{R}^W_{I_\beta}}^T\left\lfloor\mathbf{g}^W_\beta\Delta t\right\rfloor_\times B_{\mathbf{g}} \\ 0 \\ 0 \\ 0 \\ \mathbf{I} \end{bmatrix} \tag{18}$$

where $\lfloor\rfloor_\times$ represents the skew matrix of a 3D vector, and $B_{\mathbf{g}}$ corresponds to the form of $B_{\mathbf{x}}$, as described in Equation (17).

3.5. RGB-Colored Global Map

After acquiring a LiDAR scan, the system first performs motion compensation, and then, determines the color information of each point from a time-synchronized image (if available). For a LiDAR point $\mathbf{P}^L_i$, we project it onto the image using the pinhole projection model:

$$\mathbf{p}^C_i = [u_i, v_i] = \pi\left(\mathbf{T}^C_L\mathbf{P}^L_i\right) \tag{19}$$

where $\mathbf{T}^C_L$ represents the extrinsic parameters between the LiDAR and camera, and $\pi(\cdot)$ denotes the pinhole projection model. The pixel positions u_i and v_i obtained by the

projection are generally not integers, so we use linear interpolation to obtain the RGB values of the projected points:

$$\begin{aligned} r_u &= u_i - \lfloor u_i \rfloor,\ r_v = v_i - \lfloor v_i \rfloor \\ c &= r_u \times r_v \times c_{u+1,v+1} + (1 - r_u) \times (1 - r_v) \times c_{u,v} \\ &\quad + (1 - r_u) \times r_v \times c_{u,v+1} + r_u \times (1 - r_v) \times c_{u+1,v} \end{aligned} \tag{20}$$

where $\lfloor \cdot \rfloor$ represents the floor function, and c denotes the RGB value of the image color channels. The colored point cloud is then updated into the global map with the estimated LiDAR pose according to the follow criteria: (1) The time interval between the current frame and the last keyframe is greater than the pre-set threshold; (2) The change in pose of the current frame with respect to the last keyframe exceeds a certain threshold. Any LiDAR scan that meets one of these criteria is selected as a keyframe to update the global map.

4. Experiments

We implement our system in C++ using the Robot Operating System (ROS) [23] framework and validate it on both public and private datasets. The proposed method is evaluated alongside existing techniques such as SSL-SLAM [13], LeGO-LOAM [5], LIO-SAM [6], Point-LIO [19], and FAST-LIO2 [17] using public and private datasets, focusing on metrics including running time, positioning accuracy, and mapping details. A comparison of the supported LiDAR sensor types and technologies used by the open-source algorithms utilized in our experiments and our proposed algorithm is presented in Table 1.

Table 1. Comparison of the technologies used in each system.

System	Supported LiDAR	Using Inertia	Gravity Inclusion
SSL-SLAM	Solid-state	×	×
LeGO-LOAM	Mechanical	✓	×
LIO-SAM	Mechanical	✓	×
Point-LIO	Both	✓	✓
FAST-LIO2	Both	✓	✓
VE-LIOM	Both	✓	✓

The datasets used in the experiments are drawn from two categories: publicly available datasets that provide IMU and LiDAR readings, and privately collected datasets. The selected experimental scenarios encompass both indoor and outdoor environments. The public datasets include those provided in the SSL-SLAM paper, UrbanLoco [24], and the NTU VIRAL [25] dataset, details of which are presented in Table 2. The private datasets were collected using three self-designed handheld multi-sensor suites, as illustrated in Figures 4 and 5. For the RealSense L515 sensor, it can directly output colored point clouds. In order to enable Livox Mid360 and Livox Avia to also render color point clouds, we incorporated FLIR cameras into handheld suites 2 and 3. Further details are provided in Table 3 (HS stands for handheld suite).

Table 2. Details of the public dataset.

	SSL-SLAM	UrbanLoco	NTU VIRAL
LiDAR model	RealSense L515	Velodyne 32E	Ouster1-16
LiDAR type	Solid-state	Mechanical	Mechanical
IMU model	Bosch BMI085	Xsens Mti 10	VectorNav
Scenario	Warehouse	Urban road	Campus

Table 3. Details of the private dataset.

	HS 1	HS 2	HS 3
LiDAR model	RealSense L515	Livox Mid360	Livox Avia
LiDAR type	Solid-state	Hybrid	Solid-state
IMU model	Bosch BMI085	ICM40609	Bosch BMI088
Scenario	Office	Garden	Building

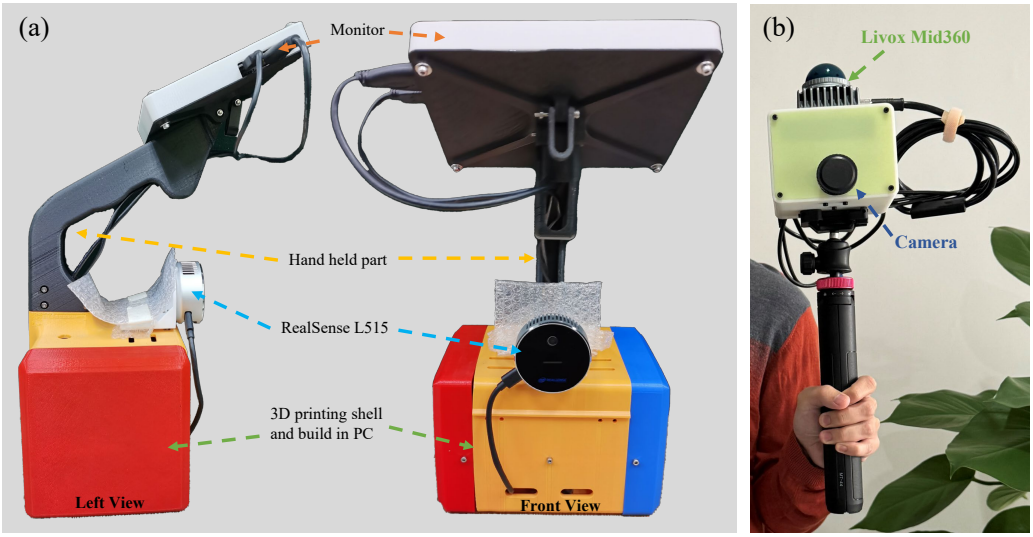


Figure 4. Handheld suite designed for RealSense L515 (**a**) and Livox Mid360 (**b**).

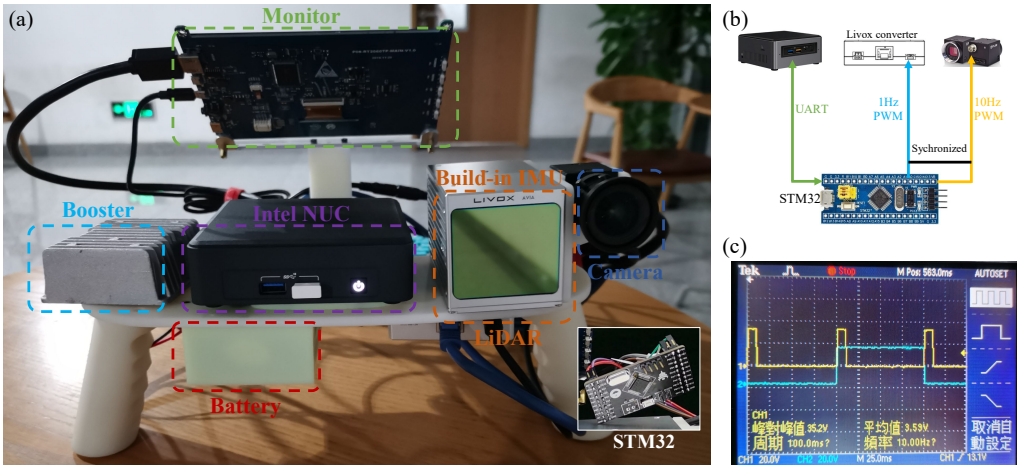


Figure 5. (**a**) Handheld suite with hardware synchronization designed for Livox Avia and FLIR camera. (**b**) The hardware synchronization diagram. (**c**) Synchronous waveforms.

The datasets used in the experiment were collected by different sensor suites, but all are tested on an Intel NUC (https://www.intel.com/content/www/us/en/products/details/nuc.html, accessed on 26 July 2024) with an Intel i7-1165G7 processor. Besides the point cloud downsampling resolution for indoor and outdoor scenarios, all other parameters in the experiments are kept consistent. Since the Intel Realsense L515 used indoors has a smaller FoV compared to the outdoor scenarios, the geometric structure features per frame are less in indoor scenes. To enhance the accuracy and robustness of indoor localization, we retained more points for indoor scenarios. We have compiled the statistics of the raw and involved points number per frame and downsampling resolution for different LiDAR sensors used in the experiments, as shown in Table 4, where the involved points refer to the points after downsampling and are used for odometry estimation. Due to the higher number of involved points in indoor scenarios, the algorithm takes more time.

Table 4. Details of the number of points per frame and downsampling resolution of different LiDAR sensors.

Section	LiDAR	Raw Points	Resolution	Involved Points
Section 4.1.2 (Indoor)	Intel Realsense L515	171,000	0.16	32,000
Section 4.2.1 (Outdoor)	Velodyne HDL-32E	57,000	0.50	5000
Section 4.2.2 (Outdoor)	OS1-16 Gen1	16,000	0.50	3800
Section 4.2.3 (Outdoor)	Livox Mid360	20,000	0.50	1500
	Livox Aian	24,000	0.50	1800

4.1. Indoor Experiments

For indoor experiments, we utilized the lightweight RealSense L515 sensor, a solid-state LiDAR with a small field of view (FoV) that integrates an IMU and an RGB camera, capable of directly outputting colored point clouds. The datasets used included publicly available datasets provided by the original SSL-SLAM paper, along with an office scene dataset collected by handheld suite 1.

4.1.1. SSL-SLAM AGV Dataset

The SSL-SLAM AGV dataset, provided by the SSL-SLAM author, was collected using an AGV (automated guided vehicle) platform operating within a warehouse. The AGV was manually controlled to navigate around the warehouse and return to its starting point. We compared the performance of our method with SSL-SLAM in terms of time consumption and accuracy. Since SSL-SLAM is a purely LiDAR-based algorithm, we disabled IMU measurements in our algorithm to ensure a fair comparison. This means that only LiDAR data were utilized for pose estimation, and our method is referred to as *VE-LOAM*. The positioning results are presented in Figure 6, and the time consumption of each module is listed in Table 5. It is evident that owing to the utilization of direct registration methods our method achieves faster processing times while maintaining comparable positioning accuracy. Specifically, we perform direct registration of raw points and eliminate the feature extraction step in the preprocessing module. Additionally, the use of the ikd-Tree structure facilitates quicker nearest-neighbor searches, thereby enhancing the efficiency of the odometry estimation module.

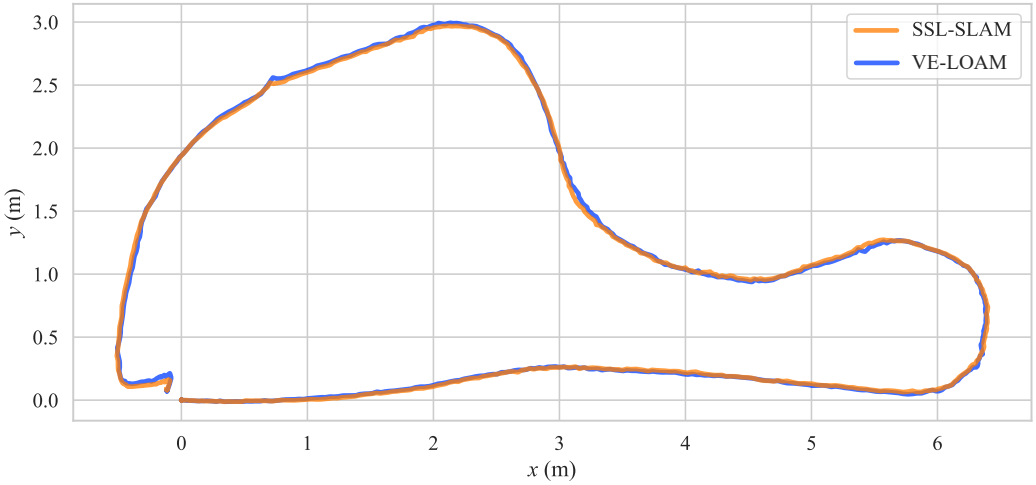


Figure 6. Comparison between proposed method and SSL-SLAM.

Table 5. The running times of two systems.

Module	Preprocessing	Odometry Estimation
SSL-SLAM (ms)	9.35	17.86
VE-LOAM (ms)	2.57	11.23

4.1.2. Handheld Suite 1 Dataset

As the AGV operates smoothly within the aforementioned dataset, it poses minimal challenges to the algorithm. To further validate the positioning accuracy and robustness of the proposed method, we designed a handheld suite for RealSense L515, depicted in Figure 4a. Unlike the vehicle platform, the handheld device experiences vibrations during data collection, and its viewing angle undergoes significant changes, presenting considerable challenges to the SLAM algorithm. Figure 7 illustrates the mapping results obtained from SSL-SLAM and VE-LIOM using data collected by the handheld device in an office setting. In this experiment, we enabled IMU measurements, thus employing the complete tightly coupled LiDAR-inertial algorithm. The estimated trajectories and mapping details indicate that the fusion of IMU data results in smoother positioning outcomes and reduced cumulative error in our algorithm. Table 6 provides a breakdown of the time consumption for each module. Although the runtime of the odometry estimation module in VE-LIOM is increased compared to that of VE-LOAM, the overall time consumption of VE-LIOM remains lower than that of SSL-SLAM. This demonstrates that VE-LIOM maintains good real-time performance.



Figure 7. Indoor localization and mapping in the office environment. Data are collected by the handheld suite as shown in Figure 4. (**a**,**e**) are the global mapping results based on SSL-SLAM and VE-LIOM, respectively. The estimated trajectories are plotted in green. (**b**) shows significant cumulative errors. (**c**,**f**) are the original camera views. (**d**,**g**) are the reconstruction details.

Table 6. The running times of three systems.

Module	Preprocessing	Odometry Estimation
SSL-SLAM (ms)	18.10	26.52
VE-LOAM (ms)	6.32	16.13
VE-LIOM (ms)	4.38	31.91

4.2. Outdoor Experiments

The outdoor experimental scenarios comprise urban roads, academic buildings, small gardens, and similar environments. Experimental data were sourced from the UrbanLoco dataset, the NTU VIRAL dataset, as well as data collected through handheld suites 2 and 3.

4.2.1. UrbanLoco Dataset

The UrbanLoco dataset is gathered from densely populated urban areas in Hong Kong and encompasses a diverse array of dynamic elements, including pedestrians, vehicles, and bicyclists. This compilation offers a realistic portrayal of the urban environments commonly encountered by autonomous vehicles. The data collection platform comprises a Velodyne HDL-32E LiDAR, an Xsens MTi-10 IMU, and a GNSS receiver. Ground truth data are provided by the Novatel SPAN-CPT, a navigation system that integrates real-time kinematic (RTK)-corrected GNSS signals and IMU measurements.

We conduct a comparative analysis of our method with LeGO-LOAM, LIO-SAM, and FAST-LIO2 utilizing the UrbanLoco dataset. The positioning trajectories are depicted in Figure 8, with the mean translation error (MTE) summarized in Table 7. In the table,

the best and second-best results are highlighted in **bold** and underlined, respectively. Our findings reveal that VE-LIOM achieves the highest positioning accuracy in the HK_3 sequence and the second-best accuracy in the HK_1, HK_2, and HK_4 sequences. Notably, for the HK_5 sequence, LIO-SAM failed to estimate the trajectory, while LeGO-LOAM exhibited the lowest MTE among the four algorithms. Additionally, runtime performance comparisons of the four algorithms are presented in Table 8, demonstrating VE-LIOM's consistent outperformance across all sequences. This observation suggests that VE-LIOM is computationally efficient, thus holding promise for real-time applications. In summary, while all methods demonstrated competitive performance, VE-LIOM showcases advantages in terms of accuracy across multiple sequences and computational efficiency. We illustrate the mapping results of VE-LIOM on the HK_4 sequence in Figure 9.

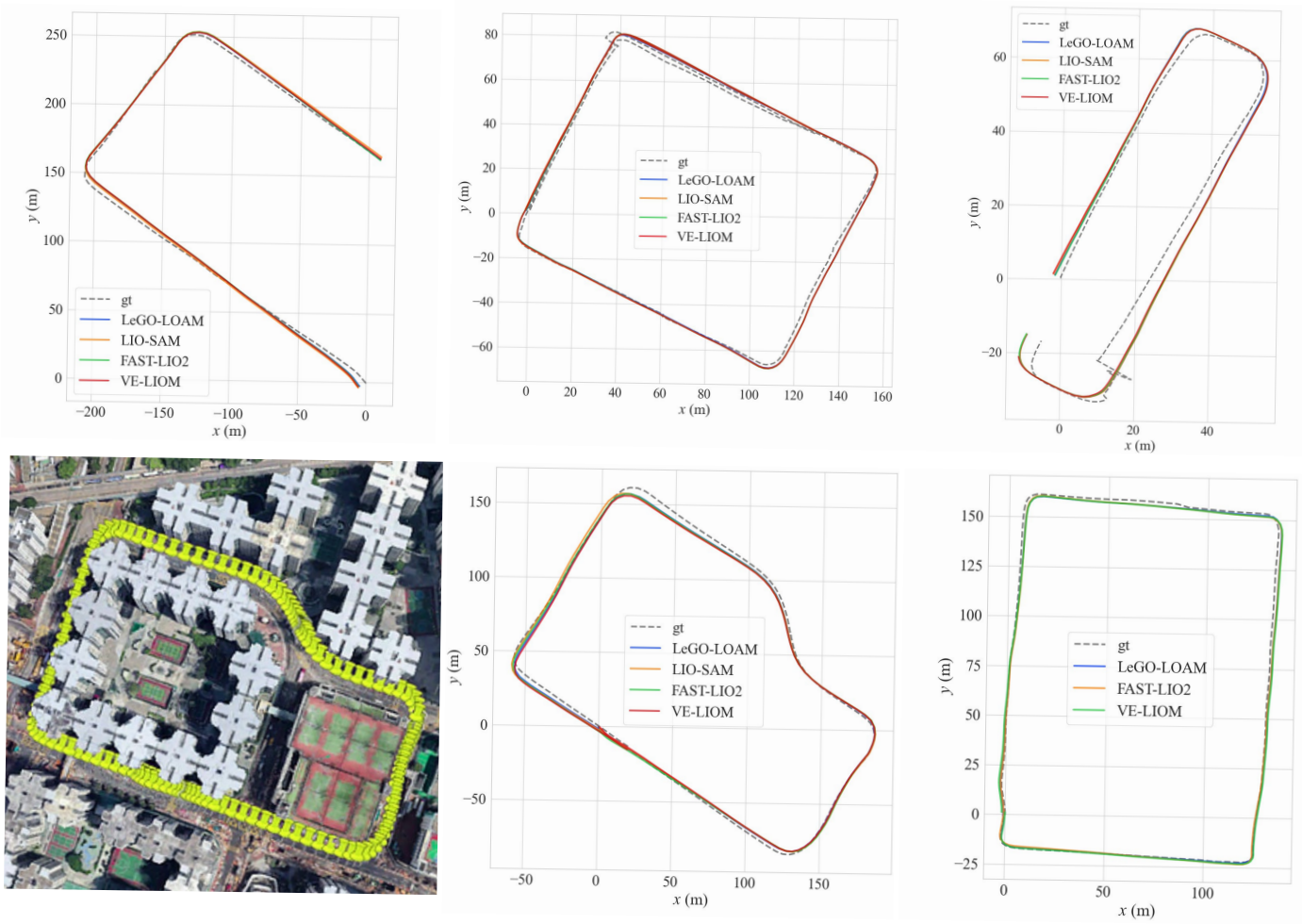


Figure 8. Trajectory estimation results of LeGO-LOAM, LIO-SAM, FAST-LIO2, and VE-LIOM on the five sequences of the UrbanLoco dataset. The gray dashed line represents the ground truth (gt) provided by GNSS. The subgraph in the bottom left corner is the satellite image of the HK_4 sequence.

Table 7. MTE of LeGO-LOAM, LIO-SAM, FAST-LIO2, and VE-LIOM on UrbanLoco dataset.

Sequence	LeGO-LOAM	LIO-SAM	FAST-LIO2	VE-LIOM
HK_1	**3.36 m**	4.06 m	3.92 m	3.81 m
HK_2	**1.60 m**	1.68 m	1.64 m	1.63 m
HK_3	2.50 m	2.56 m	2.51 m	**2.48 m**
HK_4	**2.88 m**	3.27 m	3.30 m	3.19 m
HK_5	**1.41 m**	Fail	1.72 m	1.85 m

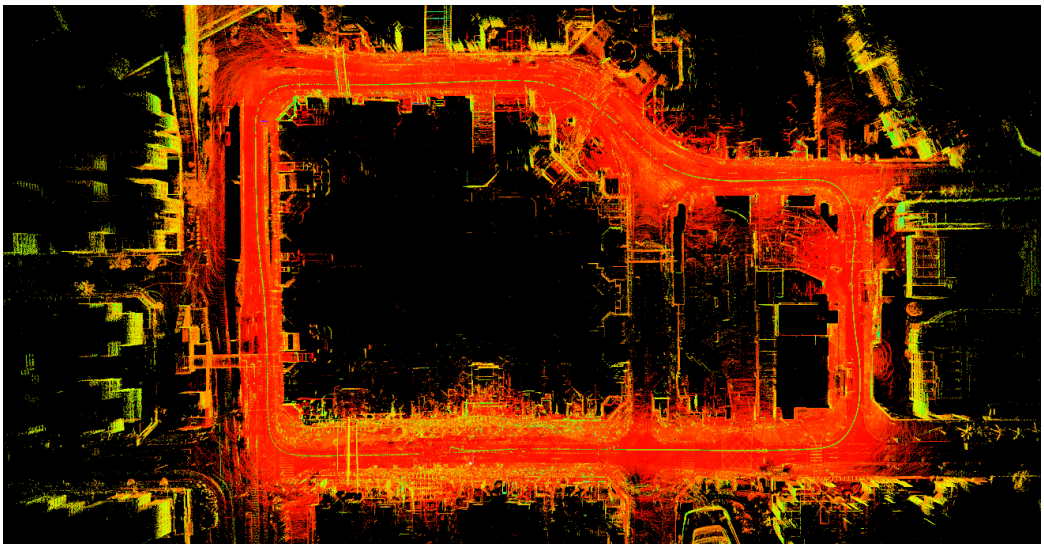


Figure 9. Mapping results of VE-LIOM on HK_4 sequence.

Table 8. Runtimes of LeGO-LOAM, LIO-SAM, FAST-LIO2, and VE-LIOM on UrbanLoco dataset.

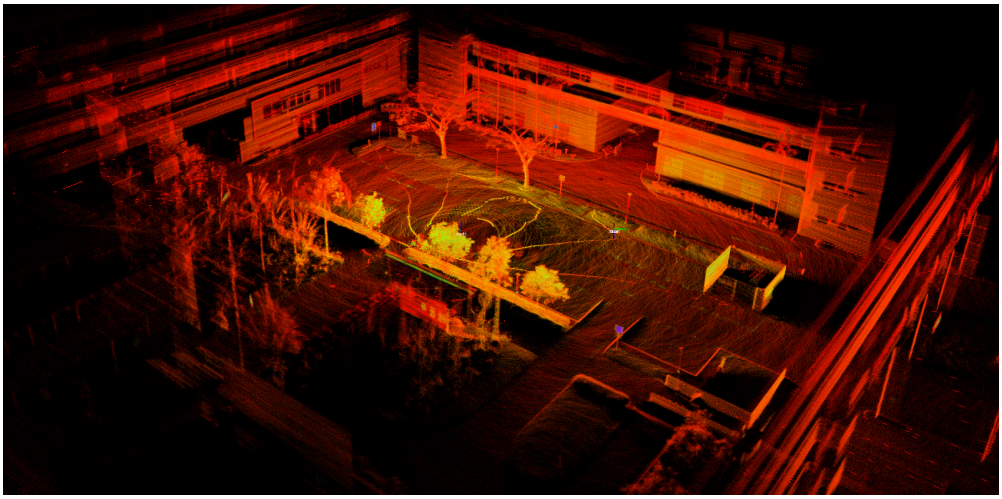
Sequence	LeGO-LOAM	LIO-SAM	FAST-LIO2	VE-LIOM
HK_1	48.33 ms	50.44 ms	18.18 ms	**15.97 ms**
HK_2	49.29 ms	50.29 ms	13.39 ms	**9.98 ms**
HK_3	40.92 ms	47.22 ms	12.72 ms	**9.61 ms**
HK_4	54.96 ms	52.61 ms	18.81 ms	**13.32 ms**
HK_5	34.35 ms	Fail	11.59 ms	**8.62 ms**

4.2.2. NTU VIRAL Dataset

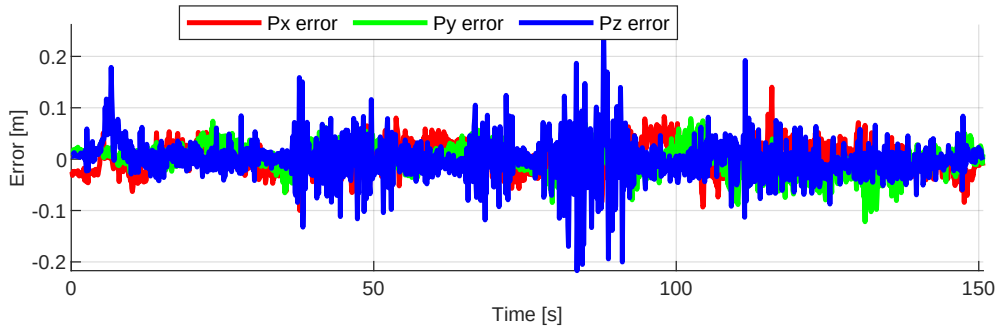
The NTU VIRAL dataset was acquired using an unmanned aerial vehicle (UAV) platform. It integrates two OS1-16 Gen1 LiDARs and an external IMU (VectorNav VN100, 385 Hz). This dataset encompasses both indoor and outdoor environments. We conducted a comparative analysis of the position absolute trajectory error (P-ATE) and runtime performance of LIO-SAM (https://github.com/hku-mars/FAST_LIO), and VE-LIOM against ground truth using the evaluation tool (https://ntu-aris.github.io/ntu_viral_dataset/evaluation_tutorial.html) provided by the dataset's authors. The summarized results are presented in Tables 9 and 10. Among these nine sequences, the *nya* sequences are derived from indoor environments, while the remainder originate from outdoor settings. It is worth noting that LIO-SAM relies on 9-axis IMU data for its operation, whereas FAST-LIO2 and VE-LIOM require only 6-axis IMU data. To ensure fairness in the comparison, we disabled the GPS fusion and loop closure functionalities in LIO-SAM. All three algorithms exclusively utilized the horizontally mounted LiDAR and VectorNav IMU data provided within the dataset. Consequently, the localization results of LIO-SAM deviate slightly from those reported in the original dataset paper [25], particularly evident in the sbs_01 and sbs_03 sequences, where LIO-SAM localization exhibited significant drift.

In Table 9, the optimal odometry result is denoted in **bold**, while the second best is underlined. Additionally, Table 10 provides a summary of the average runtime per frame for three algorithms across nine dataset sequences. The time consumption encompasses processing durations for data preprocessing, odometry estimation, and mapping. Drawing from the findings presented in Tables 9 and 10, several conclusions emerge regarding the performance of LIO-SAM, FAST-LIO2, and VE-LIOM on the NTU VIRAL dataset. In terms of P-ATE, FAST-LIO2 achieved the best performance on the eee_01 sequence, with an error of 0.131 m, while VE-LIOM outperformed the other two methods on the eee_03, nya_01, nya_02, nya_03, sbs_01, and sbs_03 sequences. LIO-SAM showed the best performance on the eee_02 and sbs_02 sequences. However, it is worth noting that LIO-SAM had significantly higher errors on the sbs_01 and sbs_03 sequences. Interestingly, VE-LIOM excelled

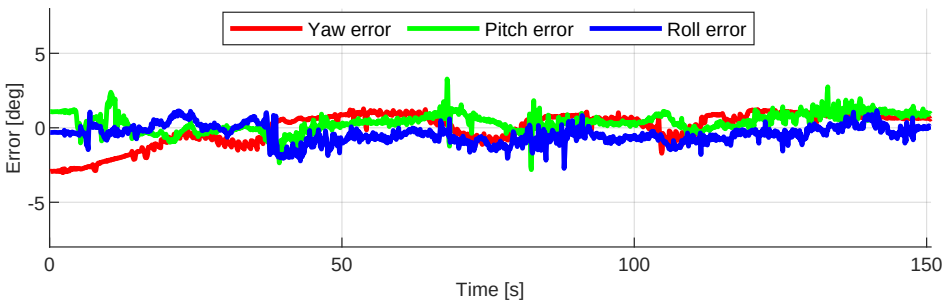
particularly on the nya sequences. This can likely be attributed to the indoor environment of the nya sequences, which includes more horizontal and vertical points compared to the predominantly outdoor scenes of other sequences. This environment facilitates more accurate point cloud classification optimization, thereby enhancing VE-LIOM's performance in these scenarios. Regarding runtime performance, VE-LIOM consistently outperformed LIO-SAM and FAST-LIO2 across all sequences, with the lowest runtime recorded at 7.64 ms on the nya_01 sequence. This suggests that VE-LIOM is more computationally efficient, which is a crucial factor for real-time applications. In summary, while all three methods showcase competitive performance, VE-LIOM exhibits advantages in both accuracy in several sequences and computational efficiency. We present the mapping results and corresponding positioning errors of VE-LIOM on the eee_03 sequence in Figure 10.



(**a**)



(**b**)



(**c**)

Figure 10. The mapping result (**a**), position error (**b**), and rotation error (**c**) of VE-LIOM on NTU VIRAL datatset sequence eee_03.

Table 9. P-ATE of LIO-SAM, FAST-LIO2, and VE-LIOM on NTU VIRAL dataset.

Sequence	LIO-SAM	FAST-LIO2	VE-LIOM
eee_01	0.321 m	**0.131 m**	0.232 m
eee_02	**0.067 m**	0.124 m	0.349 m
eee_03	0.104 m	0.163 m	**0.055 m**
nya_01	0.077 m	0.122 m	**0.044 m**
nya_02	0.089 m	0.142 m	**0.059 m**
nya_03	0.084 m	0.144 m	**0.059 m**
sbs_01	675.259 m	0.142 m	**0.061 m**
sbs_02	**0.084 m**	0.140 m	0.102 m
sbs_03	1257.159 m	0.133 m	**0.107 m**

Table 10. Comparison of runtime performance of LIO-SAM, FAST-LIO2, and VE-LIOM on NTU VIRAL dataset.

Sequence	LIO-SAM	FAST-LIO2	VE-LIOM
eee_01	78.07 ms	17.44 ms	**11.49 ms**
eee_02	54.08 ms	12.29 ms	**10.13 ms**
eee_03	56.08 ms	12.71 ms	**9.13 ms**
nya_01	51.38 ms	10.68 ms	**7.64 ms**
nya_02	53.79 ms	11.55 ms	**7.81 ms**
nya_03	48.28 ms	10.16 ms	**8.08 ms**
sbs_01	46.84 ms	10.62 ms	**7.81 ms**
sbs_02	55.99 ms	10.70 ms	**8.06 ms**
sbs_03	63.29 ms	13.35 ms	**8.11 ms**

4.2.3. Ablation Study

To validate the effectiveness of the proposed weighted point-to-plane registration and online gravity estimation method, we conducted an ablation study using data collected with handheld suites 2 and 3, as illustrated in Figures 4b and 5. Six sets of data were gathered at multiple locations near the new main building and the front garden of Beihang University using handheld devices equipped with Mid-360 and Livox Avia sensors, respectively. During data collection, experimenters walked in a loop while holding the devices, ensuring the starting and ending points were aligned to facilitate evaluation of loop closure cumulative positioning errors for different methods. A representative localization and mapping result was selected to demonstrate the impact of enabling the weighted point-to-plane registration and online gravity estimation features, shown in Figure 11. Disabling these proposed features resulted in noticeable cumulative positioning errors, whereas activating them led to a closer alignment of the estimated trajectory's starting and ending points. This observation underscores the effectiveness of the proposed method in enhancing localization accuracy.

Furthermore, we compared our method with Point-LIO and FAST-LIO2. The version of VE-LIOM with the gravity online estimation disabled is denoted as VE-LIOM$_G$, and the version with the weighted point-to-plane registration disabled is denoted as VE-LIOM$_W$. Table 11 presents the cumulative loop closure errors of these algorithms. In the handheld suite 2 dataset, FAST-LIO2 achieved the best performance in the garden_01 sequence with an error of 0.13 m, while VE-LIOM outperformed the other methods in the garden_02 and garden_03 sequences with errors of 1.26 m and 0.57 m, respectively. Point-LIO, however, had significantly higher errors across all sequences in this dataset. In the handheld suite 3 dataset, Point-LIO achieved the best performance in the building_01 sequence with an error of 0.03 m. VE-LIOM outperformed the other methods in the building_02 and building_03 sequences with errors of 1.33 m and 1.39 m, respectively. FAST-LIO2, however, had significantly higher errors across all sequences in this dataset. In summary, while all the methods demonstrated competitive performance, VE-LIOM showed advantages in

terms of accuracy in several sequences. Point-LIO and FAST-LIO2, however, had higher errors in certain sequences.

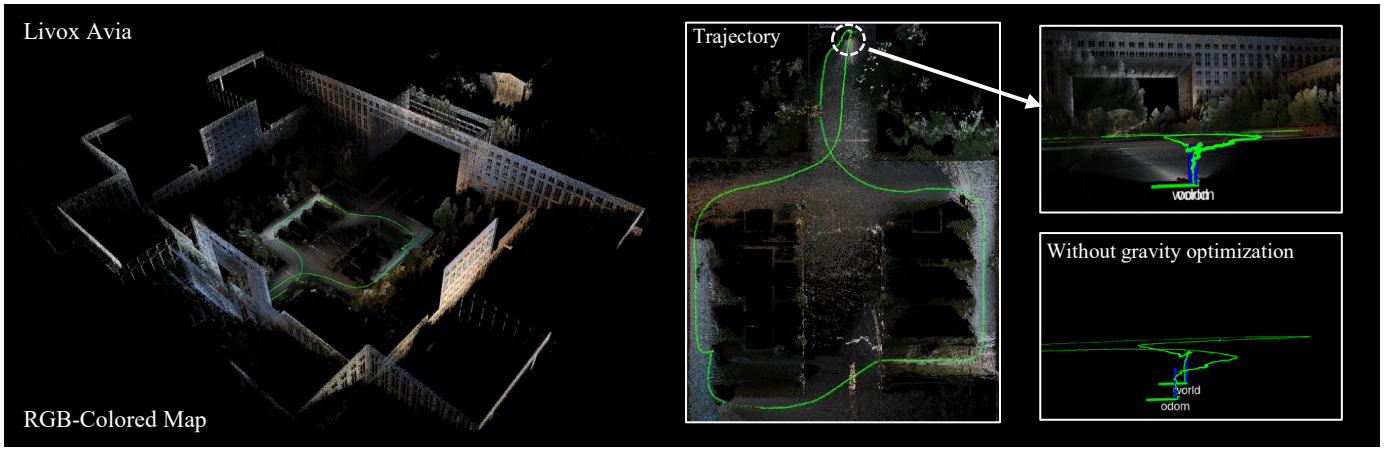


Figure 11. Mapping and pose estimation results of VE-LIOM on building_01 sequence. We employed time-synchronized images to render the point cloud map, yielding an RGB-colored map. In the absence of gravity optimization, cumulative errors were observed in the localization results; yet, with the activation of gravity optimization, the trajectory was effectively closed.

From the results presented in Table 11, we can observe the effectiveness of gravity optimization by comparing VE-LIOM$_G$ and VE-LIOM. For handheld suite 2, VE-LIOM consistently outperforms VE-LIOM$_G$ in all sequences. For instance, in the garden_01 sequence, VE-LIOM achieves an error of 0.26 m, which is lower than the 0.32 m error of VE-LIOM$_G$. Similar trends can be observed in the garden_02 and garden_03 sequences. The same pattern is observed for handheld suite 3. In all sequences, VE-LIOM shows lower errors compared to VE-LIOM$_G$. For example, in the building_01 sequence, VE-LIOM achieves an error of 0.05 m, compared to the 0.10 m error of VE-LIOM$_G$. When comparing VE-LIOM$_W$ to VE-LIOM, it is evident that the weighted point-to-plane registration method enhances the accuracy. For instance, in the garden_02 sequence, VE-LIOM$_W$ achieves an error of 1.87 m, whereas VE-LIOM achieves a lower error of 1.26 m. This trend is consistent across other sequences as well, such as garden_03 and building_03. These results clearly demonstrate the effectiveness of the proposed weighted point-to-plane registration and online gravity estimation method in VE-LIOM. By incorporating the proposed methods, VE-LIOM achieves more accurate results compared to VE-LIOM$_G$ and VE-LIOM$_W$. Although Point-LIO and FAST-LIO2 also estimate gravity as a state variable, in our experimental scenarios the presence of a large amount of leaves and grass introduces noise and hinders the fitting of local planes. The weighted point-to-plane registration method we employ effectively suppresses the influence of this noise.

Table 11. Accumulated error of different methods.

Dataset	Sequence	Point- LIO	FAST-LIO2	VE-LIOM$_G$	VE-LIOM$_W$	VE-LIOM
HS 2	garden_01	0.60 m	0.13 m	0.32 m	0.38 m	0.26 m
	garden_02	3.86 m	3.37 m	1.51 m	1.87 m	1.26 m
	garden_03	3.73 m	0.76 m	0.66 m	0.72 m	0.57 m
HS 3	building_01	0.03 m	0.11 m	0.10 m	0.07 m	0.05 m
	building_02	2.57 m	2.64 m	1.96 m	1.65 m	1.33 m
	building_03	5.86 m	3.43 m	2.11 m	1.83 m	1.39 m

5. Conclusions

In this paper, we introduce an efficient and tightly integrated LiDAR-inertial odometry and mapping system. To accommodate various types of LiDAR, we employ direct point

cloud registration methods. For improved positioning accuracy and efficiency, we propose a weighted point-to-plane fitting and classification method. Additionally, our algorithm supports online gravity optimization, mitigating potential error accumulation. We evaluate our proposed system on multiple publicly available datasets and our custom-designed handheld platforms, encompassing vehicles, drones, and handheld devices equipped with five different brands of LiDAR. Test scenarios include urban roads, campuses, warehouses, and office environments. The results across diverse platforms and scenarios demonstrate the versatility of our method with various types of LiDAR, providing reliable and precise positioning and mapping results in diverse environments.

For future work, we plan to integrate visual and GNSS information to develop a more robust and accurate positioning, navigation, and mapping system, thereby expanding its potential applications and impact in navigation and mapping technologies.

Author Contributions: Conceptualization, Y.G. and L.Z.; methodology, Y.G.; software, Y.G.; supervision, L.Z.; writing—original draft, Y.G.; writing—review and editing, L.Z. All authors have read and agreed to the published version of the manuscript.

Funding: The project is supported by the National Natural Science Foundation of China (Grant No. 42274037), the Aeronautical Science Foundation of China (Grant No. 2022Z022051001), and the National key research and development program of China (Grant No. 2020YFB0505804).

Data Availability Statement: Data available on request due to restrictions eg privacy or ethical.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Zhao, J.; He, X.; Li, J.; Feng, T.; Ye, C.; Xiong, L. Automatic vector-based road structure mapping using multibeam LiDAR. *Remote Sens.* **2019**, *11*, 1726. [CrossRef]
2. Zhang, H.; Yu, X.; Ha, S.; Westerlund, T. LiDAR-Generated Images Derived Keypoints Assisted Point Cloud Registration Scheme in Odometry Estimation. *Remote Sens.* **2023**, *15*, 5074. [CrossRef]
3. Pang, C.; Zhou, L.; Huang, X. A Low-Cost 3D SLAM System Integration of Autonomous Exploration Based on Fast-ICP Enhanced LiDAR-Inertial Odometry. *Remote Sens.* **2024**, *16*, 1979. [CrossRef]
4. Zhang, J.; Singh, S. LOAM: Lidar Odometry and Mapping in Real-time. *Robot. Sci. Syst.* **2014**, *2*, 1–9.
5. Shan, T.; Englot, B. Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain. In Proceedings of the 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Madrid, Spain, 1–5 October 2018; pp. 4758–4765.
6. Shan, T.; Englot, B.; Meyers, D.; Wang, W.; Ratti, C.; Rus, D. Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping. In Proceedings of the 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Las Vegas, NV, USA, 24 October–24 January 2021; pp. 5135–5142.
7. Tagliabue, A.; Tordesillas, J.; Cai, X.; Santamaria-Navarro, A.; How, J.P.; Carlone, L.; Agha-mohammadi, A. LION: Lidar-Inertial observability-aware navigator for Vision-Denied environments. In *Proceedings of the International Symposium on Experimental Robotics*; Springer: Cham, Switzerland, 2020; pp. 380–390.
8. Xu, W.; Zhang, F. Fast-lio: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter. *IEEE Robot. Autom. Lett.* **2021**, *6*, 3317–3324. [CrossRef]
9. Wang, H.; Wang, C.; Chen, C.; Xie, L. F-LOAM: Fast LiDAR Odometry and Mapping. In Proceedings of the 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Prague, Czech Republic, 27 September–1 October 2021.
10. Ye, H.; Chen, Y.; Liu, M. Tightly coupled 3d lidar inertial odometry and mapping. In Proceedings of the 2019 International Conference on Robotics and Automation (ICRA), Montreal, QC, Canada, 20–24 May 2019; pp. 3144–3150.
11. Forster, C.; Carlone, L.; Dellaert, F.; Scaramuzza, D. On-Manifold Preintegration for Real-Time Visual–Inertial Odometry. *IEEE Trans. Robot.* **2016**, *33*, 1–21. [CrossRef]
12. Qin, C.; Ye, H.; Pranata, C.E.; Han, J.; Zhang, S.; Liu, M. LINS: A Lidar-Inertial State Estimator for Robust and Efficient Navigation. In Proceedings of the 2020 IEEE International Conference on Robotics and Automation (ICRA), Paris, France, 31 May–31 August 2020; pp. 8899–8906.
13. Wang, H.; Wang, C.; Xie, L. Lightweight 3-D localization and mapping for solid-state LiDAR. *IEEE Robot. Autom. Lett.* **2021**, *6*, 1801–1807. [CrossRef]
14. Lin, J.; Zhang, F. Loam livox: A fast, robust, high-precision LiDAR odometry and mapping package for LiDARs of small FoV. In Proceedings of the 2020 IEEE International Conference on Robotics and Automation (ICRA), Paris, France, 31 May–31 August 2020; pp. 3126–3131.

15. Livox-SDK. LIO-Livox: A Robust LiDAR-Inertial Odometry for Livox LiDAR. 2024. Available online: https://github.com/Livox-SDK/LIO-Livox (accessed on 12 July 2024).
16. Li, K.; Li, M.; Hanebeck, U.D. Towards high-performance solid-state-lidar-inertial odometry and mapping. *IEEE Robot. Autom. Lett.* **2021**, *6*, 5167–5174. [CrossRef]
17. Xu, W.; Cai, Y.; He, D.; Lin, J.; Zhang, F. Fast-lio2: Fast direct lidar-inertial odometry. *IEEE Trans. Robot.* **2022**, *38*, 2053–2073. [CrossRef]
18. Cai, Y.; Xu, W.; Zhang, F. ikd-Tree: An incremental KD tree for robotic applications. *arXiv* **2021**, arXiv:2102.10808.
19. He, D.; Xu, W.; Chen, N.; Kong, F.; Yuan, C.; Zhang, F. Point-LIO: Robust High-Bandwidth Light Detection and Ranging Inertial Odometry. *Adv. Intell. Syst.* **2023**, *5*, 2200459. [CrossRef]
20. Qin, T.; Li, P.; Shen, S. Vins-mono: A robust and versatile monocular visual-inertial state estimator. *IEEE Trans. Robot.* **2018**, *34*, 1004–1020. [CrossRef]
21. Mur-Artal, R.; Tardós, J.D. Visual-inertial monocular SLAM with map reuse. *IEEE Robot. Autom. Lett.* **2017**, *2*, 796–803. [CrossRef]
22. Hertzberg, C.; Wagner, R.; Frese, U.; Schröder, L. Integrating generic sensor fusion algorithms with sound state representations through encapsulation of manifolds. *Inf. Fusion* **2013**, *14*, 57–77. [CrossRef]
23. Quigley, M.; Conley, K.; Gerkey, B.; Faust, J.; Foote, T.; Leibs, J.; Wheeler, R.; Ng, A.Y. ROS: An open-source Robot Operating System. In Proceedings of the ICRA Workshop on Open Source Software, Kobe, Japan, 12–17 May 2009; Volume 3, p. 5.
24. Wen, W.; Zhou, Y.; Zhang, G.; Fahandezh-Saadi, S.; Bai, X.; Zhan, W.; Tomizuka, M.; Hsu, L.T. UrbanLoco: A full sensor suite dataset for mapping and localization in urban scenes. In Proceedings of the 2020 IEEE International Conference on Robotics and Automation (ICRA), Paris, France, 31 May–31 August 2020; pp. 2310–2316.
25. Nguyen, T.M.; Yuan, S.; Cao, M.; Lyu, Y.; Nguyen, T.H.; Xie, L. Ntu viral: A visual-inertial-ranging-lidar dataset, from an aerial vehicle viewpoint. *Int. J. Robot. Res.* **2022**, *41*, 270–280. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.